Ohjelmistotuotanto

Syksy 2026

Luento 6

10.11.2026

Kurssipalaute

- Kurssipalaute
  - Kurssilla lopussa kerättävän palautteen lisäksi ns. jatkuva palaute https://norppa.helsinki.fi

Paja ja Discord

- Apua tehtäviin pajassa BK107
  - ma 16-19
  - ti 13-16
  - to 16-19
  - pe 13-16
- Kurssin Discord: https://study.cs.helsinki.fi/discord/join/ohtu
  - kanavalle liitytään komennolla */join course TKT20006 - Ohjelmistotuotanto - ohtu*

Miniprojektit

- Käynnissä!
- Aloitustilaisuudet
  - ~~ma 9.11. klo 14–16 C220~~
  - ti 10.11. klo 14–16 D122
  - ke 11.11. klo 16–18 C220
  - to 12.11. klo 10–12 BK114
  - to 12.11. klo 14–16 B222
- Seuraavat viikot: sprinttien katselmointi ja suunnittelu samassa aikaikkunassa
- Loppudemot
  - **ti 8.12. klo 14–16 B123**
  - **ke 9.12. klo 10–12 B123**

Testauksen tasot

- *Yksikkötestaus* (unit testing)
  - Yksittäisten luokkien, metodien ja moduulien testaus erillään muusta kokonaisuudesta
- *Integraatiotestaus* (integration testing)
  - Yksittäin testattujen komponenttien liittäminen yhteen eli integrointi ja kokonaisuuden testaus
- *Järjestelmätestaus* (system testing)
  - Toimiiko ohjelmisto vaatimuksiin kirjatulla tavalla?
  - Tutkii järjestelmää kokonaisuudessaan: *end to end -testaus*
  - Jakautuu useisiin alalajeihin
- *Käyttäjän hyväksymistestaus* (user acceptance testing)
  - Loppukäyttäjän tuotteelle suorittama testaus

Testauksen tasot

- *Yksikkötestaus* (unit testing)
  - Yksittäisten luokkien, metodien ja moduulien testaus erillään muusta kokonaisuudesta
- *Integraatiotestaus* (integration testing)
  - Yksittäin testattujen komponenttien liittäminen yhteen eli integrointi ja kokonaisuuden testaus
- *Järjestelmätestaus* (system testing)
  - Toimiiko ohjelmisto vaatimuksiin kirjatulla tavalla?
  - Tutkii järjestelmää kokonaisuudessaan: *end to end -testaus*
  - Jakautuu useisiin alalajeihin
- *Käyttäjän hyväksymistestaus* (user acceptance testing)
  - Loppukäyttäjän tuotteelle suorittama testaus
- Regressiotestaus

Ketterien menetelmien testauskäytänteet

Ketterien menetelmien testauskäytänteet

- Sprintin aikana toteutettavat ominaisuudet integroidaan muuhun koodiin sekä testataan

Ketterien menetelmien testauskäytänteet

- Sprintin aikana toteutettavat ominaisuudet integroidaan muuhun koodiin sekä testataan
- Sykli ominaisuuden määrittelystä sen ohjelmointiin ja testaukseen on erittäin lyhyt

Ketterien menetelmien testauskäytänteet

- Sprintin aikana toteutettavat ominaisuudet integroidaan muuhun koodiin sekä testataan
- Sykli ominaisuuden määrittelystä sen ohjelmointiin ja testaukseen on erittäin lyhyt
- Testausta sprintin “ensimmäisestä päivästä” lähtien, testaus integroitu suunnitteluun ja toteutukseen

Ketterien menetelmien testauskäytänteet

- Sprintin aikana toteutettavat ominaisuudet integroidaan muuhun koodiin sekä testataan
- Sykli ominaisuuden määrittelystä sen ohjelmointiin ja testaukseen on erittäin lyhyt
- Testausta sprintin “ensimmäisestä päivästä” lähtien, testaus integroitu suunnitteluun ja toteutukseen
- Automatisointi erittäin tärkeässä roolissa
  - testejä suoritetaan usein

Testaajat osana kehitystiimiä

- Ideaalitilanteessa testaajia sijoitettu kehittäjätiimiin, myös ohjelmoijat kirjoittavat testejä
  - tiimit *cross functional*

Testaajat osana kehitystiimiä

- Ideaalitilanteessa testaajia sijoitettu kehittäjätiimiin, myös ohjelmoijat kirjoittavat testejä
  - tiimit *cross functional*
- Testaajan rooli: *virheiden etsijästä virheiden estäjään*
  - testaaja auttaa tiimiä kirjoittamaan automatisoituja testejä, jotka pyrkivät estämään bugien pääsyn koodiin
  - *build quality in*

Ketterien menetelmien testauskäytänteitä

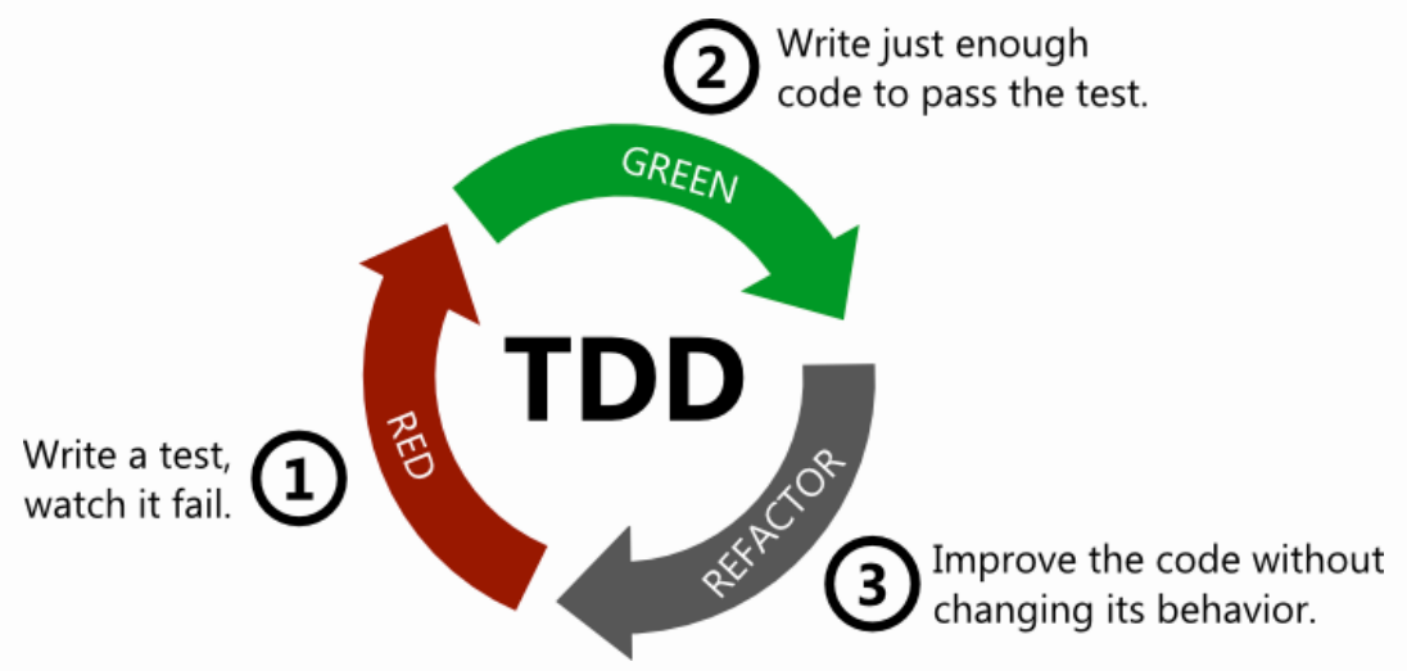
Tänään aiheena

Ketterien menetelmien testauskäytänteitä

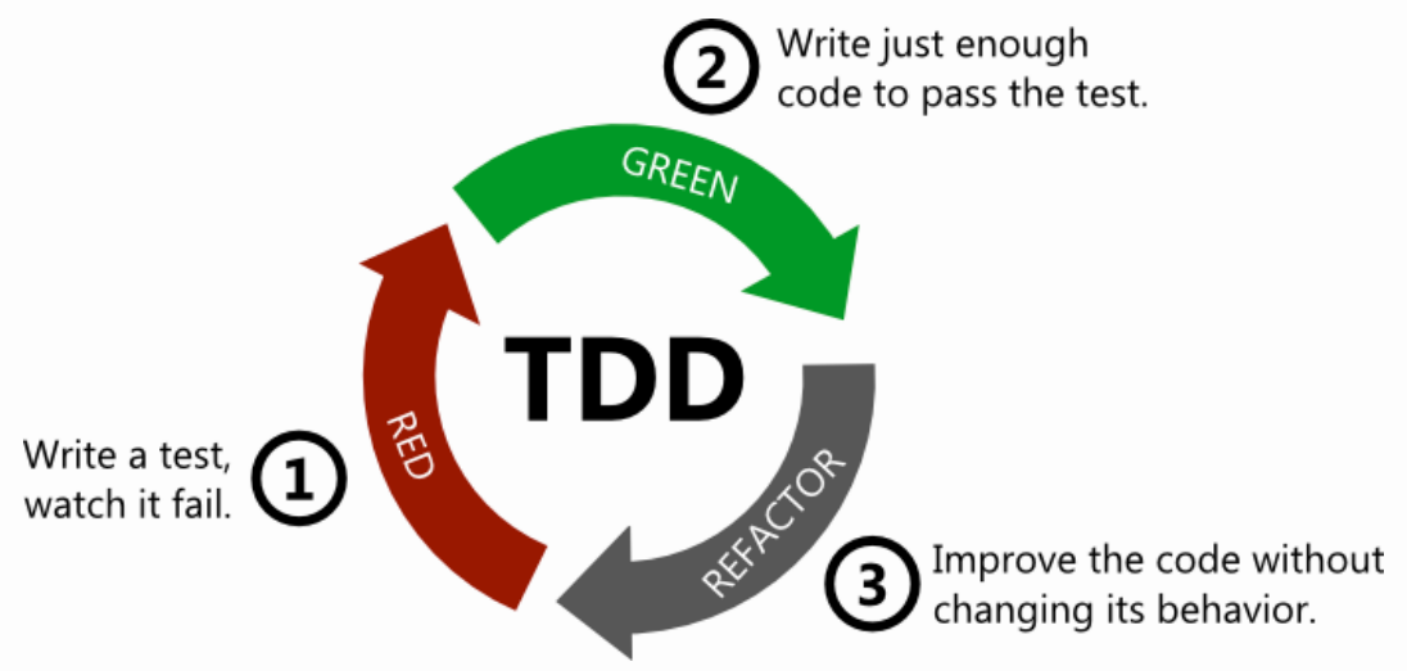
Tänään aiheena

- Test driven development (TDD)
- User storyjen tasolla tapahtuva automatisoitu testaus
- Continuous Integration (CI) eli jatkuva integraatio
- Exploratory testing, suomeksi tutkiva testaus
- Tuotannossa tapahtuva testaus

Test driven development (TDD)

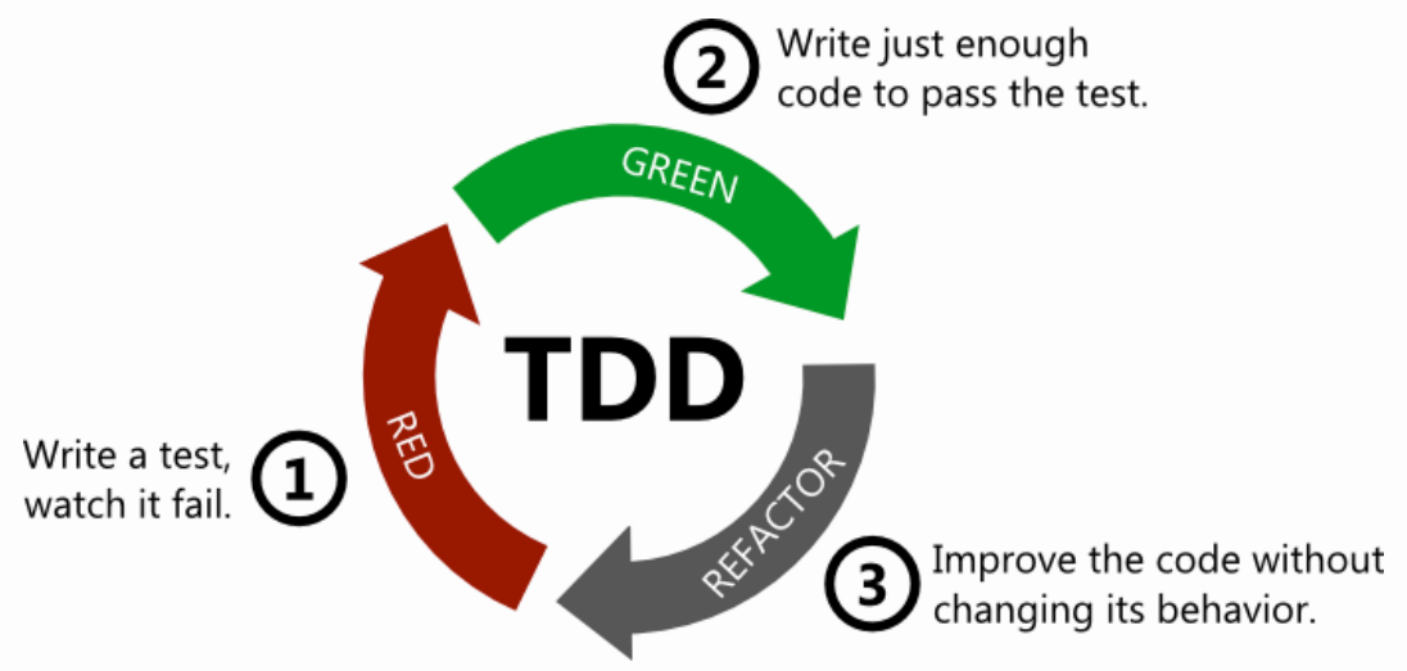


Test driven development (TDD)



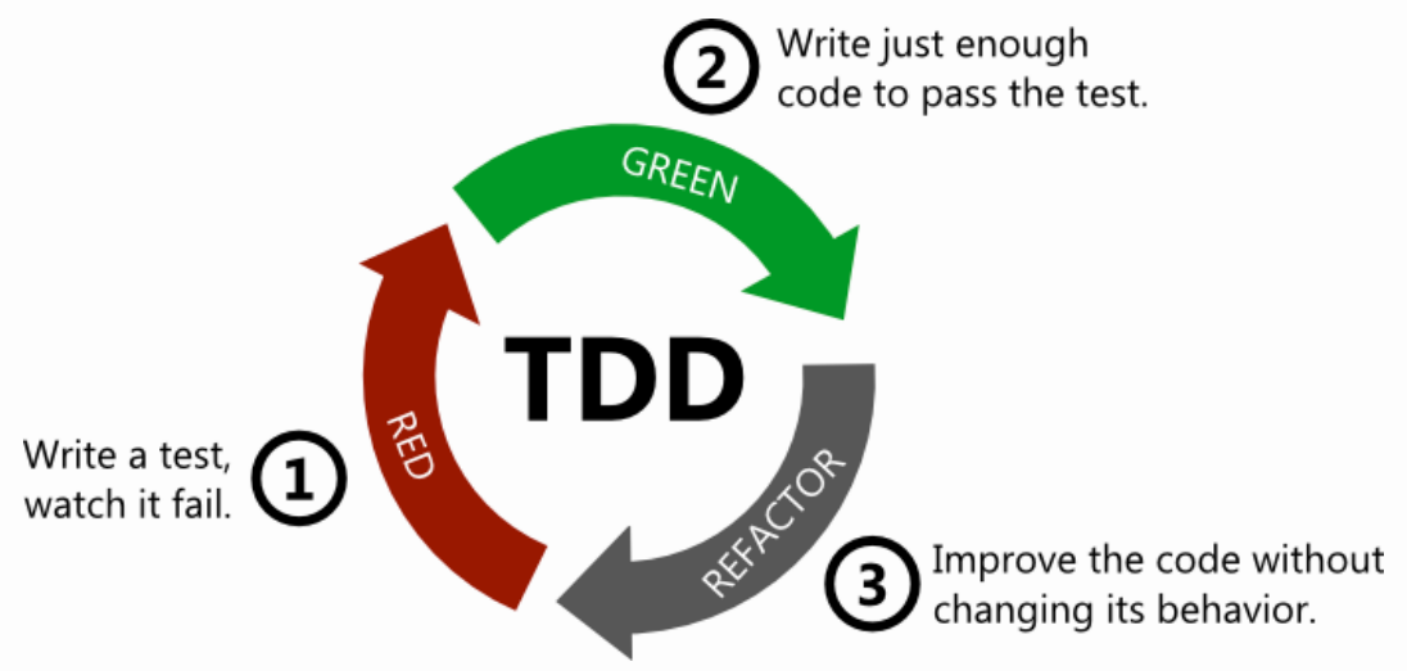
1. Kirjoitetaan testi, joka ei mene läpi

Test driven development (TDD)



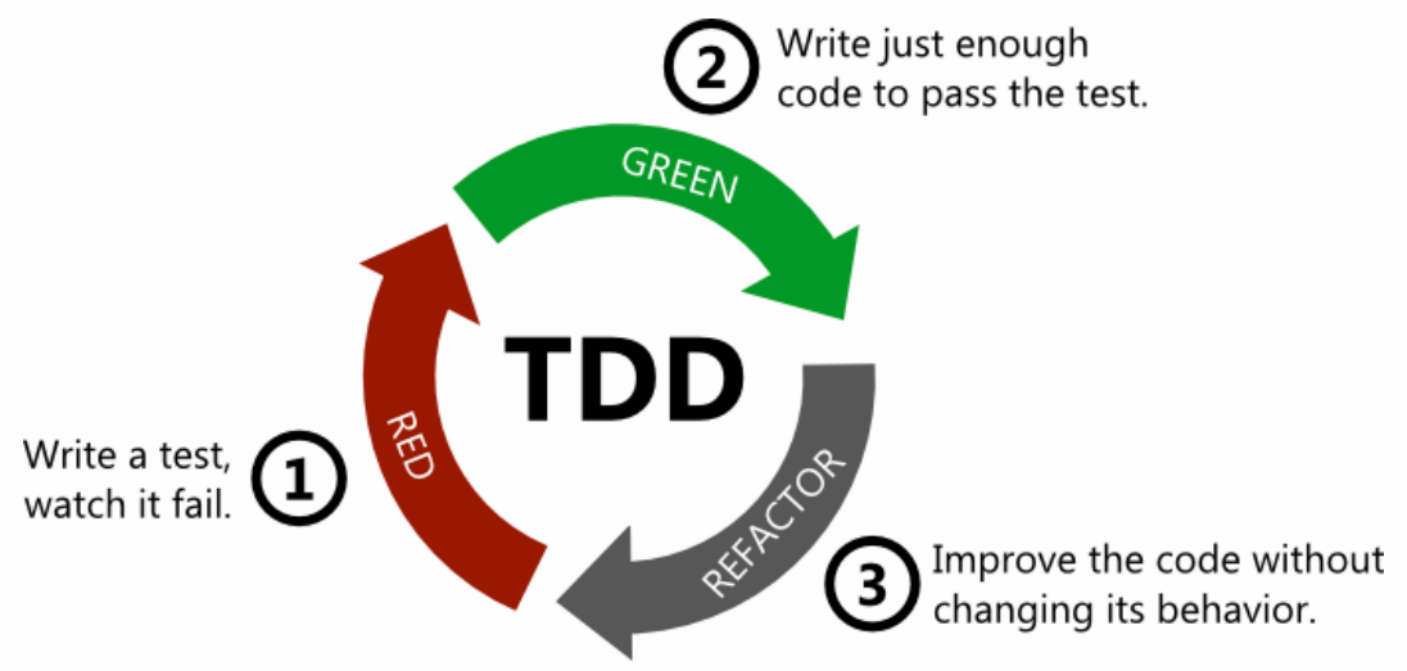
1. Kirjoitetaan testi, joka ei mene läpi
2. Kirjoitetaan koodia sen verran, että testi menee läpi

Test driven development (TDD)



1. Kirjoitetaan testi, joka ei mene läpi
2. Kirjoitetaan koodia sen verran, että testi menee läpi
3. Jos huomataan koodin rakenteen menneen huonoksi, refaktoroidaan

Test driven development (TDD)



1. Kirjoitetaan testi, joka ei mene läpi
2. Kirjoitetaan koodia sen verran, että testi menee läpi
3. Jos huomataan koodin rakenteen menneen huonoksi, refaktoroidaan
4. Jatketaan askeleesta 1

Test driven development (TDD)

- Yksi XP:n käytänteistä, Kent Beckin kehittämä
  - Joskus käytössä *tests first development*

Test driven development (TDD)

- Yksi XP:n käytänteistä, Kent Beckin kehittämä
  - Joskus käytössä *tests first development*
- TDD:llä ohjelmoitaessa toteutettavaa komponenttia ei yleensä ole tapana suunnitella tyhjentävästi etukäteen

Test driven development (TDD)

- Yksi XP:n käytänteistä, Kent Beckin kehittämä
  - Joskus käytössä *tests first development*
- TDD:llä ohjelmoitaessa toteutettavaa komponenttia ei yleensä ole tapana suunnitella tyhjentävästi etukäteen
- Testit kirjoitetaan ensisijaisesti ajatellen komponentin käyttöä
  - huomio komponentin rajapinnassa ja sen helppokäyttöisyydessä
  - ei niinkään komponentin sisäisessä toteutuksessa

Test driven development (TDD)

- Yksi XP:n käytänteistä, Kent Beckin kehittämä
  - Joskus käytössä *tests first development*
- TDD:llä ohjelmoitaessa toteutettavaa komponenttia ei yleensä ole tapana suunnitella tyhjentävästi etukäteen
- Testit kirjoitetaan ensisijaisesti ajatellen komponentin käyttöä
  - huomio komponentin rajapinnassa ja sen helppokäyttöisyydessä
  - ei niinkään komponentin sisäisessä toteutuksessa
- Komponentin sisäinen rakenne muotoutuu refaktorointien kautta

Test driven development (TDD)

- Yksi XP:n käytänteistä, Kent Beckin kehittämä
  - Joskus käytössä *tests first development*
- TDD:llä ohjelmoitaessa toteutettavaa komponenttia ei yleensä ole tapana suunnitella tyhjentävästi etukäteen
- Testit kirjoitetaan ensisijaisesti ajatellen komponentin käyttöä
  - huomio komponentin rajapinnassa ja sen helppokäyttöisyydessä
  - ei niinkään komponentin sisäisessä toteutuksessa
- Komponentin sisäinen rakenne muotoutuu refaktorointien kautta
- *"Ensin testataan, sitten koodataan, suunnitellaan vasta lopussa"*

- TDD:ssä korostetaan lopputuloksen yksinkertaisuutta

- TDD:ssä korostetaan lopputuloksen yksinkertaisuutta
- Toteutetaan toiminnallisuutta vain sen verran, mitä testien läpimeno edellyttää
  - Ei toteuteta ”varalta” ekstratoiminnallisuutta
  - *Simplicity – the art of maximizing the amount of work not done – is essential*

- TDD:ssä korostetaan lopputuloksen yksinkertaisuutta
- Toteutetaan toiminnallisuutta vain sen verran, mitä testien läpimeno edellyttää
  - Ei toteuteta ”varalta” ekstratoiminnallisuutta
  - *Simplicity – the art of maximizing the amount of work not done – is essential*
- Koodista on vaikea tehdä testattavaa jos se ei ole modulaarista ja löyhästi kytketyistä komponenteista koostuvaa
  - koodista laadukasta ylläpidettävyyden ja laajennettavuuden kannalta

- TDD:ssä korostetaan lopputuloksen yksinkertaisuutta
- Toteutetaan toiminnallisuutta vain sen verran, mitä testien läpimeno edellyttää
  - Ei toteuteta ”varalta” ekstratoiminnallisuutta
  - *Simplicity – the art of maximizing the amount of work not done – is essential*
- Koodista on vaikea tehdä testattavaa jos se ei ole modulaarista ja löyhästi kytketyistä komponenteista koostuvaa
  - koodista laadukasta ylläpidettävyyden ja laajennettavuuden kannalta
- Muita TDD:n hyviä puolia:
  - Rohkaisee ottamaan pieniä askelia kerrallaan ja toimimaan fokusoidusti
  - Virheet havaitaan nopeasti suuren testijoukon takia

TDD:llä on myös ikävät puolensa

- Testikoodia tulee paljon, usein suunnilleen saman verran kuin varsinaista koodia
  - Toisaalta TDD:llä tehty tuotantokoodi on usein hieman normaalisti tehtyä koodia lyhempi

TDD:llä on myös ikävät puolensa

- Testikoodia tulee paljon, usein suunnilleen saman verran kuin varsinaista koodia
  - Toisaalta TDD:llä tehty tuotantokoodi on usein hieman normaalisti tehtyä koodia lyhempi
- Jos ja kun koodi muuttuu, tulee testejä ylläpitää
- Legacy-koodin laajentaminen TDD:llä voi olla haastavaa

TDD:llä on myös ikävät puolensa

- Testikoodia tulee paljon, usein suunnilleen saman verran kuin varsinaista koodia
  - Toisaalta TDD:llä tehty tuotantokoodi on usein hieman normaalisti tehtyä koodia lyhempi
- Jos ja kun koodi muuttuu, tulee testejä ylläpitää
- Legacy-koodin laajentaminen TDD:llä voi olla haastavaa
- Tutkimusnäyttö TDD:n hyödyistä vähäistä
  - laatu paranee hieman, vaikutus tuottavuuteen epäselvä
  - hyödyt näyttävät tulevan pienin askelin etenemisestä ja tiheästä testaamisesta, ei niinkään testin kirjoittamisesta ennen koodia

TDD:llä on myös ikävät puolensa

- Testikoodia tulee paljon, usein suunnilleen saman verran kuin varsinaista koodia
  - Toisaalta TDD:llä tehty tuotantokoodi on usein hieman normaalisti tehtyä koodia lyhempi
- Jos ja kun koodi muuttuu, tulee testejä ylläpitää
- Legacy-koodin laajentaminen TDD:llä voi olla haastavaa
- Tutkimusnäyttö TDD:n hyödyistä vähäistä
  - laatu paranee hieman, vaikutus tuottavuuteen epäselvä
  - hyödyt näyttävät tulevan pienin askelin etenemisestä ja tiheästä testaamisesta, ei niinkään testin kirjoittamisesta ennen koodia
- Saanut uutta merkitystä tekoälyagenttien myötä: testit ovat tarkka tapa kertoa agentille, mitä koodin halutaan tekevän

Storyjen testaaminen

Storyjen testaaminen

- User storyn käsite pitää sisällään *hyväksymiskriteerit*

Storyjen testaaminen

- User storyn käsite pitää sisällään *hyväksymiskriteerit*
- Storyn *asiakas voi lisätä tuotteen ostoskoriin* eräs hyväksymiskriteeri voisi olla
  - ollessaan tuotelistauksessa ja valitessaan tuotteen jota on varastossa, menee tuote ostoskoriin
  - ja ostoskorin hinta sekä korissa olevien tuotteiden määrä päivittyy oikein

Storyjen testaaminen

- User storyn käsite pitää sisällään *hyväksymiskriteerit*
- Storyn *asiakas voi lisätä tuotteen ostoskoriin* eräs hyväksymiskriteeri voisi olla
  - ollessaan tuotelistauksessa ja valitessaan tuotteen jota on varastossa, menee tuote ostoskoriin
  - ja ostoskorin hinta sekä korissa olevien tuotteiden määrä päivittyy oikein
- Hyväksymiskriteereistä saadaan muodostettua suurin osa ohjelmiston järjestelmätason toiminnallisista testeistä

Järjestelmätestauksen automatisointi

- Hyväksymiskriteerit on tarkoituksenmukaista kirjoittaa heti storyn toteuttavan sprintin alussa
  - yhteistyössä kehitystiimin ja product ownerin kesken
  - asiakkaan kielellä, käyttämättä teknistä jargonia

Järjestelmätestauksen automatisointi

- Hyväksymiskriteerit on tarkoituksenmukaista kirjoittaa heti storyn toteuttavan sprintin alussa
  - yhteistyössä kehitystiimin ja product ownerin kesken
  - asiakkaan kielellä, käyttämättä teknistä jargonia
- Ideaalitilanteessa storyjen hyväksymiskriteereistä tehdään automaattisesti suoritettavia

Järjestelmätestauksen automatisointi

- Hyväksymiskriteerit on tarkoituksenmukaista kirjoittaa heti storyn toteuttavan sprintin alussa
  - yhteistyössä kehitystiimin ja product ownerin kesken
  - asiakkaan kielellä, käyttämättä teknistä jargonia
- Ideaalitilanteessa storyjen hyväksymiskriteereistä tehdään automaattisesti suoritettavia
- Olemassa monia työkaluja
  - eräs suosituista, erityisesti Suomessa, on suomalainen Python-pohjainen *Robot framework*
  - maailmalla tunnetumpia mm. Cucumber ja Playwright

Käyttäjähallinnan tarjoama palvelu

- Palvelun vaatimukset määrittelevät user storyt
  - A new user account can be created if a proper unused username and a proper password are given
  - User can log in with a valid username/password-combination

Käyttäjähallinnan tarjoama palvelu

- Palvelun vaatimukset määrittelevät user storyt
  - A new user account can be created if a proper unused username and a proper password are given
  - User can log in with a valid username/password-combination
- Robot-frameworkia käytettäessä jokaisesta storystä kirjoitetaan .robot- päätteinen tiedosto
  - sisältää joukon storyyn liittyvä hyväksymistestejä

Käyttäjähallinnan tarjoama palvelu

- Palvelun vaatimukset määrittelevät user storyt
  - A new user account can be created if a proper unused username and a proper password are given
  - User can log in with a valid username/password-combination
- Robot-frameworkia käytettäessä jokaisesta storystä kirjoitetaan .robot- päätteinen tiedosto
  - sisältää joukon storyyn liittyvä hyväksymistestejä
- Storyn hyväksymätestit kirjoitetaan hyödyntäen *avainsanoja*

Testit asiakkan kielellä

```
*** Test Cases ***
Run | Debug | Run in Interactive Console
Login With Correct Credentials
    Set Username  kalle
    Set Password  kalle123
    Submit Credentials
    Login Should Succeed

Run | Debug | Run in Interactive Console
Login With Incorrect Password
    Set Username  kalle
    Set Password  wrong
    Submit Credentials
    Login Should Fail With Message  Invalid username or password
```

- *Set username*, *Submit credentials* ym avainsanoja

Avainsanat mäpätään suoritettavaksi

- Avainsanojen määrittely toisten avainsanojen avulla

```
Login Should Fail With Message
    [Arguments]  ${message}
    Login Page Should Be Open
    Page Should Contain  ${message}

Load in Interactive Console
Submit Credentials
    Click Button  Login

Load in Interactive Console
Set Username
    [Arguments]  ${username}
    Input Text  username  ${username}
```

- osa avainsanoista Playwrightiin perustuvan Browser-kirjaston tarjoamia

Avainsanat mäpätään suoritettavaksi

- Avainsanojen määrittely toisten avainsanojen avulla

```
Login Should Fail With Message
    [Arguments]  ${message}
    Login Page Should Be Open
    Page Should Contain  ${message}

Load in Interactive Console
Submit Credentials
    Click Button  Login

Load in Interactive Console
Set Username
    [Arguments]  ${username}
    Input Text  username  ${username}
```

- osa avainsanoista Playwrightiin perustuvan Browser-kirjaston tarjoamia
- tai koodina

Ohjelmiston integraatio

Ohjelmiston integraatio

- Vesiputousmallissa toteutusvaiheen päättää integrointi
  - Yksittäin testatut komponentit yhdessä toimivaksi kokonaisuudeksi
  - Yhteistoiminnallisuus varmistetaan **integraatiotestien** avulla

Ohjelmiston integraatio

- Vesiputousmallissa toteutusvaiheen päättää integrointi
  - Yksittäin testatut komponentit yhdessä toimivaksi kokonaisuudeksi
  - Yhteistoiminnallisuus varmistetaan **integraatiotestien** avulla
- Perinteisesti integrointi on tuonut esiin paljon ongelmia

Ohjelmiston integraatio

- Vesiputousmallissa toteutusvaiheen päättää integrointi
  - Yksittäin testatut komponentit yhdessä toimivaksi kokonaisuudeksi
  - Yhteistoiminnallisuus varmistetaan **integraatiotestien** avulla
- Perinteisesti integrointi on tuonut esiin paljon ongelmia
- Suurten projektien integrointivaihe on kestänyt ennakoimattoman kauan

Ohjelmiston integraatio

- Vesiputousmallissa toteutusvaiheen päättää integrointi
  - Yksittäin testatut komponentit yhdessä toimivaksi kokonaisuudeksi
  - Yhteistoiminnallisuus varmistetaan **integraatiotestien** avulla
- Perinteisesti integrointi on tuonut esiin paljon ongelmia
- Suurten projektien integrointivaihe on kestänyt ennakoimattoman kauan
- Integrointivaiheen ongelmat ovat aiheuttaneet ohjelmaan suunnittelutason muutoksia

Ohjelmiston integraatio

- Vesiputousmallissa toteutusvaiheen päättää integrointi
  - Yksittäin testatut komponentit yhdessä toimivaksi kokonaisuudeksi
  - Yhteistoiminnallisuus varmistetaan **integraatiotestien** avulla
- Perinteisesti integrointi on tuonut esiin paljon ongelmia
- Suurten projektien integrointivaihe on kestänyt ennakoimattoman kauan
- Integrointivaiheen ongelmat ovat aiheuttaneet ohjelmaan suunnittelutason muutoksia
- **Integratiohelvetti**

Pois integraatiohelvetistä

- 90-luvulla huomattiin, että riskien minimoimiseksi integraatio kannattaa tehdä useammin kuin vain projektin lopussa

Pois integraatiohelvetistä

- 90-luvulla huomattiin, että riskien minimoimiseksi integraatio kannattaa tehdä useammin kuin vain projektin lopussa
- Muodostui uusi paras käytänne: *daily build* ja *smoke test*
  - The *smoke test* should exercise the entire system from end to end
  - It does not have to be exhaustive
  - but it should be capable of exposing major problems

Pois integraatiohelvetistä

- 90-luvulla huomattiin, että riskien minimoimiseksi integraatio kannattaa tehdä useammin kuin vain projektin lopussa
- Muodostui uusi paras käytänne: *daily build* ja *smoke test*
  - The *smoke test* should exercise the entire system from end to end
  - It does not have to be exhaustive
  - but it should be capable of exposing major problems
- Daily buildia ja smoke testiä käytettäessä järjestelmän integraatio tehdään (ainakin jollain tarkkuustasolla) joka päivä

Pois integraatiohelvetistä

- 90-luvulla huomattiin, että riskien minimoimiseksi integraatio kannattaa tehdä useammin kuin vain projektin lopussa
- Muodostui uusi paras käytänne: *daily build* ja *smoke test*
  - The *smoke test* should exercise the entire system from end to end
  - It does not have to be exhaustive
  - but it should be capable of exposing major problems
- Daily buildia ja smoke testiä käytettäessä järjestelmän integraatio tehdään (ainakin jollain tarkkuustasolla) joka päivä
- Komponenttien yhteensopivuusongelmat huomataan nopeasti ja niiden korjaaminen helpottuu

Pois integraatiohelvetistä

- 90-luvulla huomattiin, että riskien minimoimiseksi integraatio kannattaa tehdä useammin kuin vain projektin lopussa
- Muodostui uusi paras käytänne: *daily build* ja *smoke test*
  - The *smoke test* should exercise the entire system from end to end
  - It does not have to be exhaustive
  - but it should be capable of exposing major problems
- Daily buildia ja smoke testiä käytettäessä järjestelmän integraatio tehdään (ainakin jollain tarkkuustasolla) joka päivä
- Komponenttien yhteensopivuusongelmat huomataan nopeasti ja niiden korjaaminen helpottuu
- **Tiimin moraali paranee**, kun ohjelmistosta on olemassa päivittäin kasvava toimiva versio

Päivittäisestä jatkuvaan integraatioon

- Syntyi idea toistaa integraatiota vielä päivittäistä sykliäkin useammin: *jatkuva integraatio eli continuous integration*
  - eräs XP:n käytenäteistä

Päivittäisestä jatkuvaan integraatioon

- Syntyi idea toistaa integraatiota vielä päivittäistä sykliäkin useammin: *jatkuva integraatio eli continuous integration*
  - eräs XP:n käytenäteistä
- Koodi, automatisoidut testi, konfiguraatiot ja build-skriptit pidetään keskitetyssä repositoriossa

Päivittäisestä jatkuvaan integraatioon

- Syntyi idea toistaa integraatiota vielä päivittäistä sykliäkin useammin: *jatkuva integraatio eli continuous integration*
  - eräs XP:n käytenäteistä
- Koodi, automatisoidut testi, konfiguraatiot ja build-skriptit pidetään keskitetyssä repositoriossa
- *CI-palvelin*: vastaa konfiguraatioilta mahdollisimman läheisesti tuotantoympäristöä
  - nykyään build suoritetaan yleensä puhtaassa, kertakäyttöisessä virtuaalikoneessa tai kontissa

Päivittäisestä jatkuvaan integraatioon

- Syntyi idea toistaa integraatiota vielä päivittäistä sykliäkin useammin: *jatkuva integraatio eli continuous integration*
  - eräs XP:n käytenäteistä
- Koodi, automatisoidut testi, konfiguraatiot ja build-skriptit pidetään keskitetyssä repositoriossa
- *CI-palvelin*: vastaa konfiguraatioilta mahdollisimman läheisesti tuotantoympäristöä
  - nykyään build suoritetaan yleensä puhtaassa, kertakäyttöisessä virtuaalikoneessa tai kontissa
- CI-palvelin tarkkailee repositoriota, muutosten tapahtuessa se hakee koodin, kääntää sen ja suorittaa testit

Päivittäisestä jatkuvaan integraatioon

- Syntyi idea toistaa integraatiota vielä päivittäistä sykliäkin useammin: *jatkuva integraatio eli continuous integration*
  - eräs XP:n käytenäteistä
- Koodi, automatisoidut testi, konfiguraatiot ja build-skriptit pidetään keskitetyssä repositoriossa
- *CI-palvelin*: vastaa konfiguraatioilta mahdollisimman läheisesti tuotantoympäristöä
  - nykyään build suoritetaan yleensä puhtaassa, kertakäyttöisessä virtuaalikoneessa tai kontissa
- CI-palvelin tarkkailee repositoriota, muutosten tapahtuessa se hakee koodin, kääntää sen ja suorittaa testit
- Jos koodi ei käänny tai testit eivät mene läpi, seurauksena poikkeustilanne joka korjattava *välittömästi*

Päivittäisestä jatkuvaan integraatioon

- Syntyi idea toistaa integraatiota vielä päivittäistä sykliäkin useammin: *jatkuva integraatio eli continuous integration*
  - eräs XP:n käytenäteistä
- Koodi, automatisoidut testi, konfiguraatiot ja build-skriptit pidetään keskitetyssä repositoriossa
- *CI-palvelin*: vastaa konfiguraatioilta mahdollisimman läheisesti tuotantoympäristöä
  - nykyään build suoritetaan yleensä puhtaassa, kertakäyttöisessä virtuaalikoneessa tai kontissa
- CI-palvelin tarkkailee repositoriota, muutosten tapahtuessa se hakee koodin, kääntää sen ja suorittaa testit
- Jos koodi ei käänny tai testit eivät mene läpi, seurauksena poikkeustilanne joka korjattava *välittömästi*
- **Integraatiosta vaivaton operaatio**: ohjelmistosta olemassa koko ajan integroitu ja testattu tuore versio

- Sovelluskehittäjä aloittaa työskentelyn hakemalla koodin uusimman version versionhallinnasta

- Sovelluskehittäjä aloittaa työskentelyn hakemalla koodin uusimman version versionhallinnasta
- Kehittäjä integroi koodinsa heti muuhun koodiin ja tekee riittävän määrän automatisoituja testejä

- Sovelluskehittäjä aloittaa työskentelyn hakemalla koodin uusimman version versionhallinnasta
- Kehittäjä integroi koodinsa heti muuhun koodiin ja tekee riittävän määrän automatisoituja testejä
- Jokainen kehittäjä integroi koodinsa muuhun koodiin mahdollisimman usein, vähintään kerran päivässä

- Sovelluskehittäjä aloittaa työskentelyn hakemalla koodin uusimman version versionhallinnasta
- Kehittäjä integroi koodinsa heti muuhun koodiin ja tekee riittävän määrän automatisoituja testejä
- Jokainen kehittäjä integroi koodinsa muuhun koodiin mahdollisimman usein, vähintään kerran päivässä
- CI rohkaisee jakamaan työn pieniin osiin, jotka saadaan testeineen valmiiksi yhden työpäivän aikana
  - **CI-työprosessin noudattaminen vaatii kurinalaisuutta**

- Sovelluskehittäjä aloittaa työskentelyn hakemalla koodin uusimman version versionhallinnasta
- Kehittäjä integroi koodinsa heti muuhun koodiin ja tekee riittävän määrän automatisoituja testejä
- Jokainen kehittäjä integroi koodinsa muuhun koodiin mahdollisimman usein, vähintään kerran päivässä
- CI rohkaisee jakamaan työn pieniin osiin, jotka saadaan testeineen valmiiksi yhden työpäivän aikana
  - **CI-työprosessin noudattaminen vaatii kurinalaisuutta**
- Laskareissa käytetty *GitHub Actions* suosituin SaaS-palveluna toimiva CI-ratkaisu
  - muita esim. GitLab CI/CD ja CircleCI

- Sovelluskehittäjä aloittaa työskentelyn hakemalla koodin uusimman version versionhallinnasta
- Kehittäjä integroi koodinsa heti muuhun koodiin ja tekee riittävän määrän automatisoituja testejä
- Jokainen kehittäjä integroi koodinsa muuhun koodiin mahdollisimman usein, vähintään kerran päivässä
- CI rohkaisee jakamaan työn pieniin osiin, jotka saadaan testeineen valmiiksi yhden työpäivän aikana
  - **CI-työprosessin noudattaminen vaatii kurinalaisuutta**
- Laskareissa käytetty *GitHub Actions* suosituin SaaS-palveluna toimiva CI-ratkaisu
  - muita esim. GitLab CI/CD ja CircleCI
- Vanhempi Jenkins edelleen laajasti käytetty erityisesti yritysten omissa ympäristöissä
  - Jenkins asennetaan yleensä omalle palvelimelle

Tauko 10 min

Deployment pipeline

- CI:tä viedään vielä askel pidemmälle

Deployment pipeline

- CI:tä viedään vielä askel pidemmälle
- Integraatioprosessiin lisätään automaattinen vienti **staging-palvelimelle**
  - Ympäristö, joka kaikin tavoin mahdollisimman lähellä tuotantoympäristöä

Deployment pipeline

- CI:tä viedään vielä askel pidemmälle
- Integraatioprosessiin lisätään automaattinen vienti **staging-palvelimelle**
  - Ympäristö, joka kaikin tavoin mahdollisimman lähellä tuotantoympäristöä
- Kun uusi versio viety staging-palvelimelle, suoritetaan sille hyväksymistestaus

Deployment pipeline

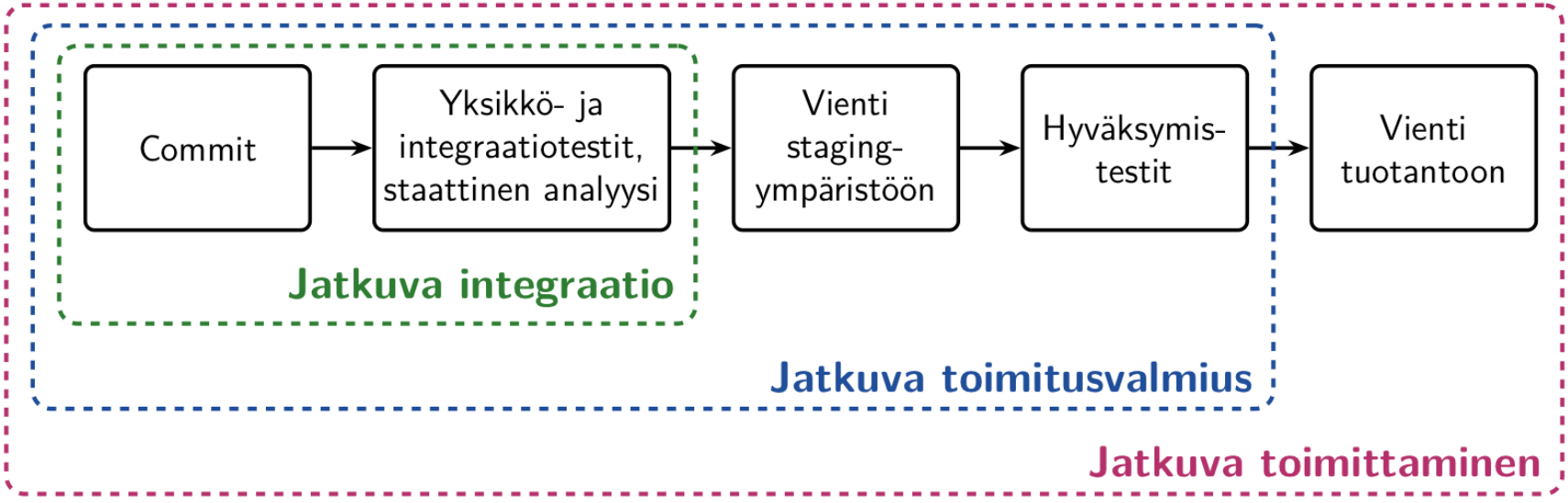
- CI:tä viedään vielä askel pidemmälle
- Integraatioprosessiin lisätään automaattinen vienti **staging-palvelimelle**
  - Ympäristö, joka kaikin tavoin mahdollisimman lähellä tuotantoympäristöä
- Kun uusi versio viety staging-palvelimelle, suoritetaan sille hyväksymistestaus
- ...jonka jälkeen siirto **tuotantopalvelimelle**

Deployment pipeline

- CI:tä viedään vielä askel pidemmälle
- Integraatioprosessiin lisätään automaattinen vienti **staging-palvelimelle**
  - Ympäristö, joka kaikin tavoin mahdollisimman lähellä tuotantoympäristöä
- Kun uusi versio viety staging-palvelimelle, suoritetaan sille hyväksymistestaus
- ...jonka jälkeen siirto **tuotantopalvelimelle**
- Parhaassa tapauksessa staging-ympäristössä tehtävien hyväksymätestien suoritus on automatisoitu
  - Ohjelmisto kulkee koko *deployment pipelinen* läpi automaattisesti

Deployment pipeline

- Vaiheet, joiden suorittaminen edellytetään, että commitattu koodi saadaan siirrettyä staging/tuotantoympäristöön



Termejä: jatkuva toimittaminen ja toimitusvalmius

- **Jatkuvasta toimittaminen engl. continuous deployment**
  - Jokainen testit läpäisevä commit päätyy automaattisesti tuotantoon

Termejä: jatkuva toimittaminen ja toimitusvalmius

- **Jatkuvasta toimittaminen engl. continuous deployment**
  - Jokainen testit läpäisevä commit päätyy automaattisesti tuotantoon
- **Jatkuvaa toimitusvalmius engl. continuous delivery**:
  - tuotantoonvientipäätös tehdään ihmisen toimesta

Termejä: jatkuva toimittaminen ja toimitusvalmius

- **Jatkuvasta toimittaminen engl. continuous deployment**
  - Jokainen testit läpäisevä commit päätyy automaattisesti tuotantoon
- **Jatkuvaa toimitusvalmius engl. continuous delivery**:
  - tuotantoonvientipäätös tehdään ihmisen toimesta
- Suuret web-palvelut julkaisevat jopa kymmeniä uusia versioita päivässä
  - Amazon, Netflix, Meta, Unity, Smartly...
  - DORA-tutkimuksessa parhaiten suoriutuvat vievät muutoksia tuotantoon tarpeen mukaan, usein useita kertoja päivässä

Tutkiva testaaminen

Tutkiva testaaminen

- Jotta järjestelmä saadaan riittävän virheettömäksi, on testaus suoritettava erittäin perusteellisesti

Tutkiva testaaminen

- Jotta järjestelmä saadaan riittävän virheettömäksi, on testaus suoritettava erittäin perusteellisesti
- Perinteinen tapa on laatia ennen testausta hyvin perinpohjainen suunnitelma
  - Jokaisesta testistä on kirjattu testisyötteet ja odotettu tulos

Tutkiva testaaminen

- Jotta järjestelmä saadaan riittävän virheettömäksi, on testaus suoritettava erittäin perusteellisesti
- Perinteinen tapa on laatia ennen testausta hyvin perinpohjainen suunnitelma
  - Jokaisesta testistä on kirjattu testisyötteet ja odotettu tulos
- Tuloksen tarkastaminen
  - Verrataan ohjelmiston toimintaa testitapaukseen kirjattuun odotettuun tulokseen

Test Scenario ID	Login-1	Test Case ID	Login-1B
Test Case Description	Login – Negative test case	Test Priority	High
Pre-Requisite	NA	Post-Requisite	NA

Test Execution Steps:

S.No	Action	Inputs	Expected Output	Actual Output	Test Browser	Test Result	Test Comments
1	Launch application	https://www.facebook.com/	Facebook home	Facebook home	IE-11	Pass	[Priya 10/17/2017 11:44 AM]: Launch successful
2	Enter invalid Email & any Password and hit login button	Email id : invalid@xyz.com Password: ******	The email address or phone number that you've entered doesn't match any account. Sign up for an account.	The email address or phone number that you've entered doesn't match any account. Sign up for an account.	IE-11	Pass	[Priya 10/17/2017 11:45 AM]: Invalid login attempt stopped

Test Scenario ID	Login-1	Test Case ID	Login-1B
Test Case Description	Login – Negative test case	Test Priority	High
Pre-Requisite	NA	Post-Requisite	NA

Test Execution Steps:

S.No	Action	Inputs	Expected Output	Actual Output	Test Browser	Test Result	Test Comments
1	Launch application	https://www.facebook.com/	Facebook home	Facebook home	IE-11	Pass	[Priya 10/17/2017 11:44 AM]: Launch successful
2	Enter invalid Email & any Password and hit login button	Email id : invalid@xyz.com Password: ******	The email address or phone number that you've entered doesn't match any account. Sign up for an account.	The email address or phone number that you've entered doesn't match any account. Sign up for an account.	IE-11	Pass	[Priya 10/17/2017 11:45 AM]: Invalid login attempt stopped

Test Scenario ID	Login-1	Test Case ID	Login-1B
Test Case Description	Login – Negative test case	Test Priority	High
Pre-Requisite	NA	Post-Requisite	NA

Test Execution Steps:

S.No	Action	Inputs	Expected Output	Actual Output	Test Browser	Test Result	Test Comments
1	Launch application	https://www.facebook.com/	Facebook home	Facebook home	IE-11	Pass	[Priya 10/17/2017 11:44 AM]: Launch successful
2	Enter invalid Email & any Password and hit login button	Email id : invalid@xyz.com Password: ******	The email address or phone number that you've entered doesn't match any account. Sign up for an account.	The email address or phone number that you've entered doesn't match any account. Sign up for an account.	IE-11	Pass	[Priya 10/17/2017 11:45 AM]: Invalid login attempt stopped

- Automatisoitujen hyväksymistestien luonne sama, syöte on tarkkaan kiinnitetty samoin kuin odotettu tuloskin

- Automatisoitujen hyväksymistestien luonne sama, syöte on tarkkaan kiinnitetty samoin kuin odotettu tuloskin
- Jos testataan vain etukäteen mietittyjen testien avulla, ei kaikkia yllättäviä tilanteita osata ennakoida

- Automatisoitujen hyväksymistestien luonne sama, syöte on tarkkaan kiinnitetty samoin kuin odotettu tuloskin
- Jos testataan vain etukäteen mietittyjen testien avulla, ei kaikkia yllättäviä tilanteita osata ennakoida
- Hyvät testaajat ovat aina tehneet virallisen dokumentoidun testauksen lisäksi epävirallista “ad hoc”-testausta

- Automatisoitujen hyväksymistestien luonne sama, syöte on tarkkaan kiinnitetty samoin kuin odotettu tuloskin
- Jos testataan vain etukäteen mietittyjen testien avulla, ei kaikkia yllättäviä tilanteita osata ennakoida
- Hyvät testaajat ovat aina tehneet virallisen dokumentoidun testauksen lisäksi epävirallista “ad hoc”-testausta
- Tästä tullut virallisesti hyväksytty testauksen muoto, kulkee nimellä *tutkiva testaaminen* (engl. exploratory testing)

Tutkiva testaaminen

- *Exploratory testing is simultaneous learning, test design and test execution*

Tutkiva testaaminen

- *Exploratory testing is simultaneous learning, test design and test execution*
- Testitapauksia ei suunnitella kattavasti etukäteen
  - Testaaja pyrkii kokemuksensa avulla löytämään järjestelmästä virheitä
  - Testaaja ohjaa toimintaansa suorittamiensa testien tuloksen perusteella

Tutkiva testaaminen

- *Exploratory testing is simultaneous learning, test design and test execution*
- Testitapauksia ei suunnitella kattavasti etukäteen
  - Testaaja pyrkii kokemuksensa avulla löytämään järjestelmästä virheitä
  - Testaaja ohjaa toimintaansa suorittamiensa testien tuloksen perusteella
- Tutkiva testaaminen ei etene sattumanvaraisesti
  - Testaussessiolle asetetaan tavoite: mitä tutkitaan ja minkälaisia virheitä etsitään

Tutkiva testaaminen

- *Exploratory testing is simultaneous learning, test design and test execution*
- Testitapauksia ei suunnitella kattavasti etukäteen
  - Testaaja pyrkii kokemuksensa avulla löytämään järjestelmästä virheitä
  - Testaaja ohjaa toimintaansa suorittamiensa testien tuloksen perusteella
- Tutkiva testaaminen ei etene sattumanvaraisesti
  - Testaussessiolle asetetaan tavoite: mitä tutkitaan ja minkälaisia virheitä etsitään
- Voi liittyä esim. muutaman user storyn toiminnallisuuteen
  - *testataan ostosten lisäystä ja poistoa ostoskorista*

Tutkiva testaaminen

- Keskeistä on kaikkien ohjelmiston tapahtuvien asioiden havainnointi
  - Etukäteen määritellyissä testeissä havainnoidaan ainoastaan reagoiko järjestelmä odotetulla tavalla

Tutkiva testaaminen

- Keskeistä on kaikkien ohjelmiston tapahtuvien asioiden havainnointi
  - Etukäteen määritellyissä testeissä havainnoidaan ainoastaan reagoiko järjestelmä odotetulla tavalla
- Kiinnitetään huomio myös varsinaisen testauksen kohteen ulkopuoleisiin asioihin
  - Klikkaillaan käyttöliittymän nappuloita epäloogisissa tilanteissa
  - Jos huomataan selaimen osoiterivillä URL https://www.webshopshop.com/ostoskori?id=10 katsotaan mitä tapahtuu jo id muutetaan käsin
  - ...

Tutkiva testaaminen

- Keskeistä on kaikkien ohjelmiston tapahtuvien asioiden havainnointi
  - Etukäteen määritellyissä testeissä havainnoidaan ainoastaan reagoiko järjestelmä odotetulla tavalla
- Kiinnitetään huomio myös varsinaisen testauksen kohteen ulkopuoleisiin asioihin
  - Klikkaillaan käyttöliittymän nappuloita epäloogisissa tilanteissa
  - Jos huomataan selaimen osoiterivillä URL https://www.webshopshop.com/ostoskori?id=10 katsotaan mitä tapahtuu jo id muutetaan käsin
  - ...
- Tietoturvan testaamisessa on monia tyypillisiä skenaariota, joita testataan tutkivan testaamisen menetelmin
  - Esim. SQL- ja JavaScript-injektiot

Tutkiva testaaminen

Tutkiva testaaminen

- Tutkiva testaaminen turhan raskas regressiotestaamisen menetelmänä
  - Testataan sprintin yhteydessä toteutettuja uusia ominaisuuksia
  - Löydettyjen virheiden toistuminen jatkossa kannattaa eliminoida tekemällä automatisoituja regressiotestejä

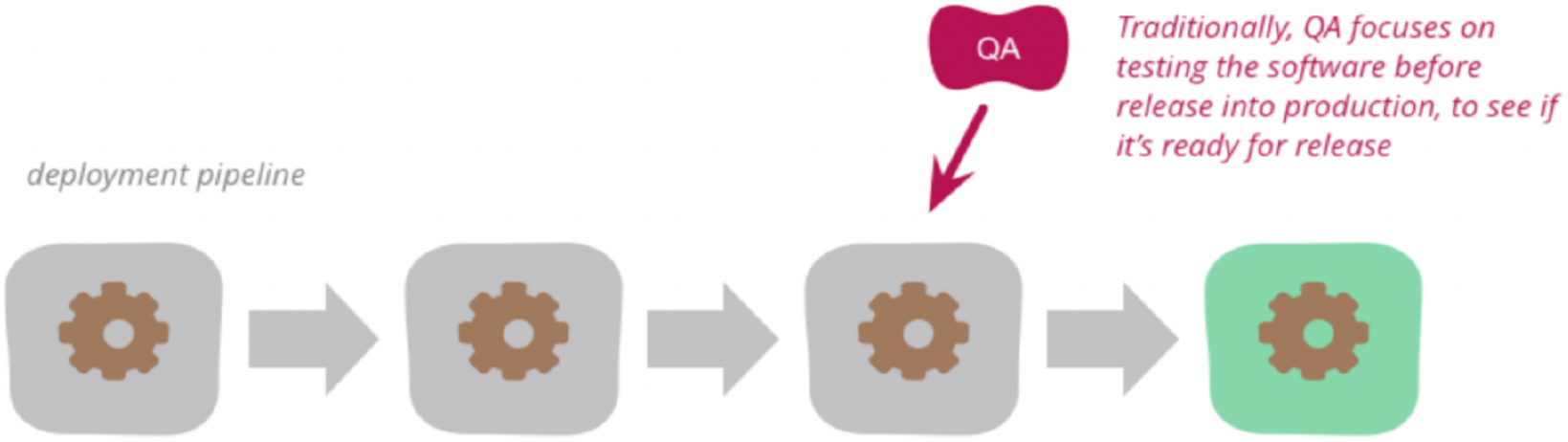
Tutkiva testaaminen

- Tutkiva testaaminen turhan raskas regressiotestaamisen menetelmänä
  - Testataan sprintin yhteydessä toteutettuja uusia ominaisuuksia
  - Löydettyjen virheiden toistuminen jatkossa kannattaa eliminoida tekemällä automatisoituja regressiotestejä
- Ei vaihtoehto normaaleille tarkkaan etukäteen määritellyille testeille vaan niitä täydentävä testauksen muoto

Tuotannossa tapahtuva testaaminen ja laadunhallinta

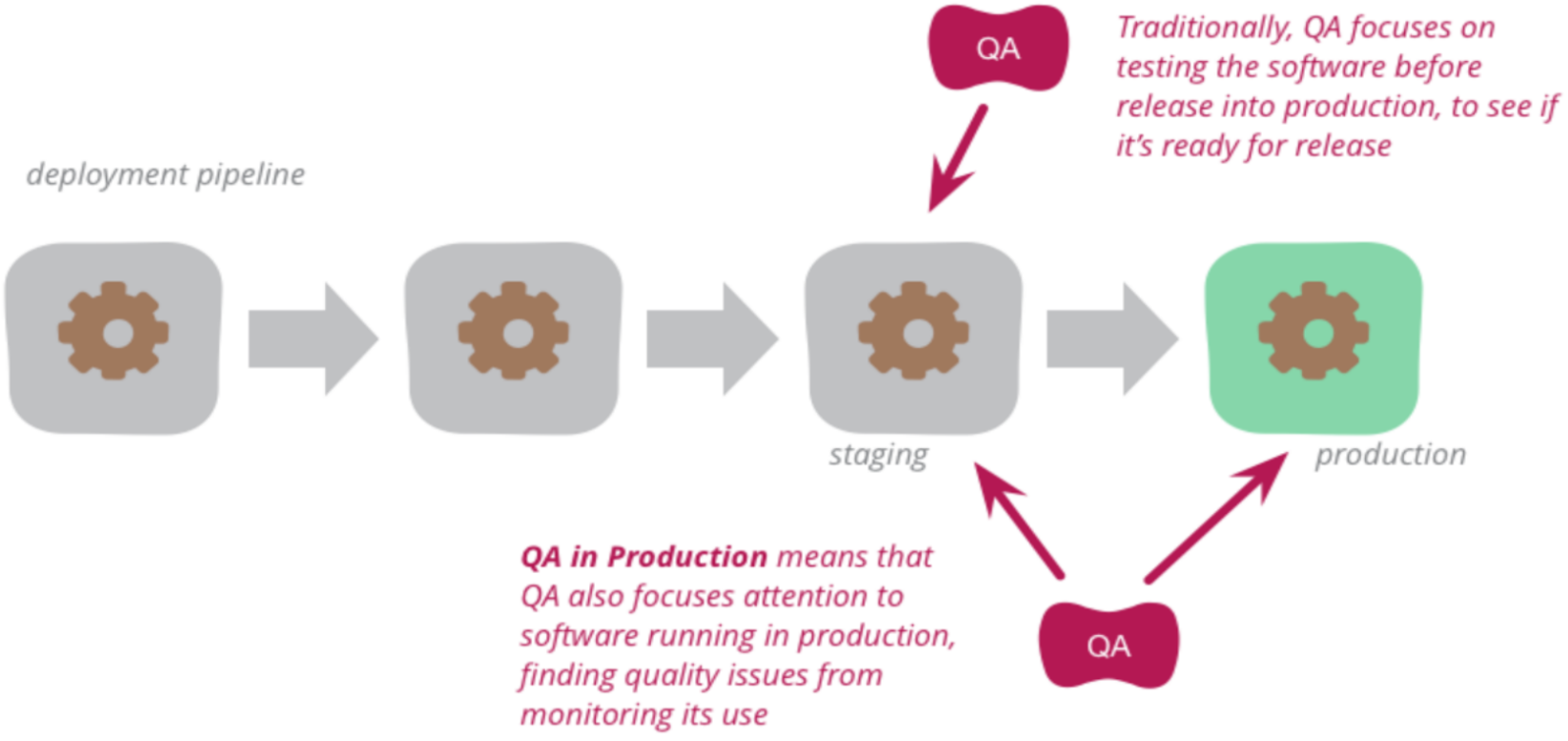
Tuotannossa tapahtuva testaaminen ja laadunhallinta

- Perinteisesti ajateltu: kaikki laadunhallintaan tehdään ennen kuin uudet toiminnallisuudet otetaan käyttöön



Tuotannossa tapahtuva testaaminen ja laadunhallinta

- Perinteisesti ajateltu: kaikki laadunhallintaan tehdään ennen kuin uudet toiminnallisuudet otetaan käyttöön



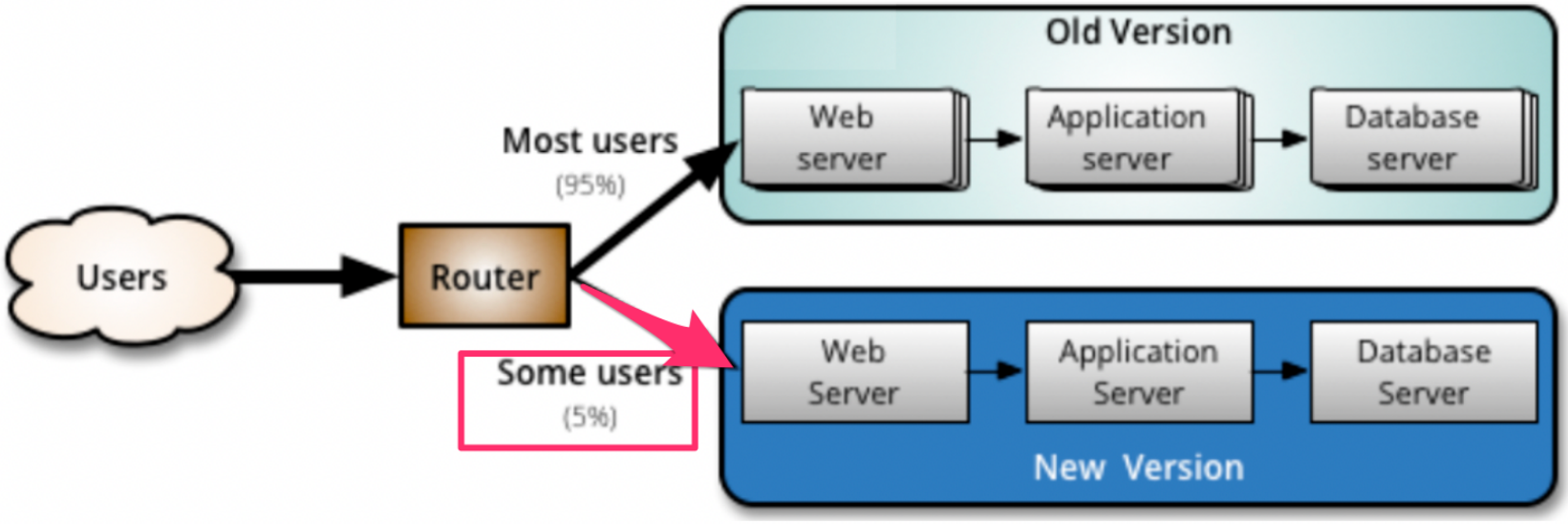
- Nykyään osa laadunhallinnasta tehdään *monitoroimalla* tuotannossa olevaa ohjelmistoa

Canary release

- Kaksi rinnakkaista tuotantoympäristöä, joista uudet ominaisuudet viedään toiseen

Canary release

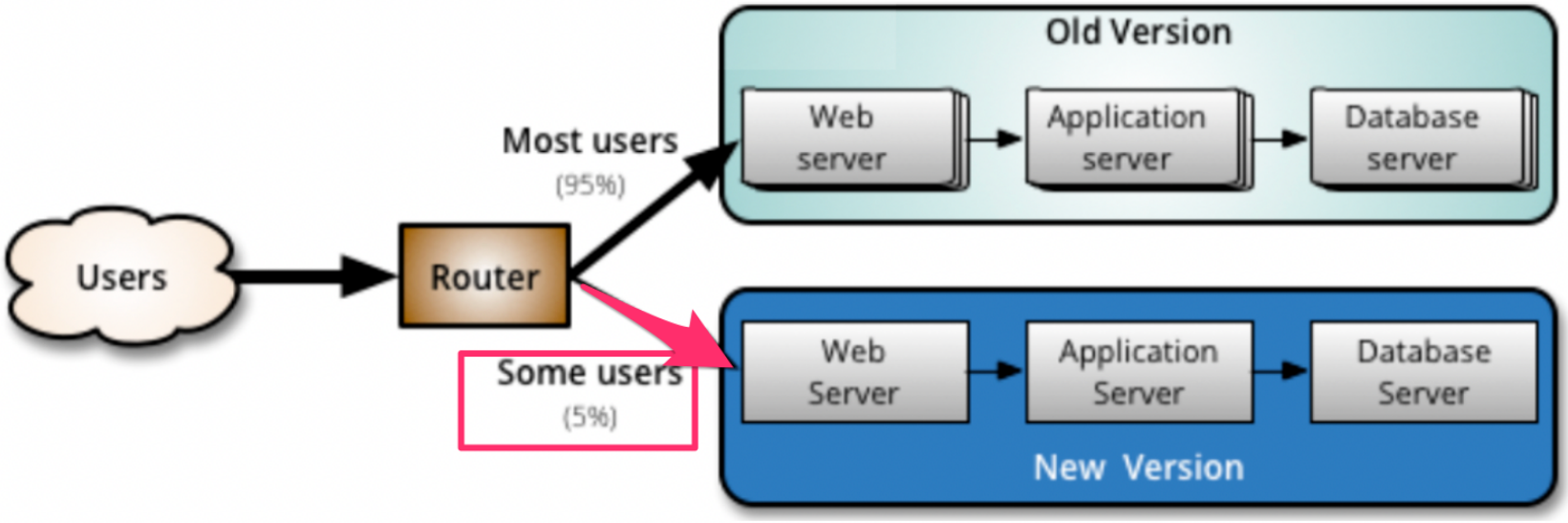
- Kaksi rinnakkaista tuotantoympäristöä, joista uudet ominaisuudet viedään toiseen



- Uuden ominaisuuden sisältävään ympäristöön ohjataan osa järjestelmän käyttäjistä

Canary release

- Kaksi rinnakkaista tuotantoympäristöä, joista uudet ominaisuudet viedään toiseen



- Uuden ominaisuuden sisältävään ympäristöön ohjataan osa järjestelmän käyttäjistä
- Uuden ominaisuuden sisältämää versiota *monitoroidaan*
  - jos ei ongelmia ohjataan kaikki liikenne uuteen versioon

- Uuden version toimivaksi varmistaminen perustuu järjestelmän *monitorointiin*

- Uuden version toimivaksi varmistaminen perustuu järjestelmän *monitorointiin*
- Esim. sosiaalisen median palvelussa
  - palvelun muistin ja prosessoriajan kulutusta
  - verkkoliikenteen määrää
  - sovelluksen eri sivujen vasteaikoja
  - kirjautuneiden käyttäjien määrää
  - luettujen ja lähetettyjen viestien määriä per käyttäjä
  - kirjautuneen käyttäjän sovelluksessa viettämää aikaa

- Uuden version toimivaksi varmistaminen perustuu järjestelmän *monitorointiin*
- Esim. sosiaalisen median palvelussa
  - palvelun muistin ja prosessoriajan kulutusta
  - verkkoliikenteen määrää
  - sovelluksen eri sivujen vasteaikoja
  - kirjautuneiden käyttäjien määrää
  - luettujen ja lähetettyjen viestien määriä per käyttäjä
  - kirjautuneen käyttäjän sovelluksessa viettämää aikaa
- Monitoroidaan palvelimen yleisen toimivuuden lisäksi *käyttäjätason metriikoita* (engl. business level metrics)

- Uuden version toimivaksi varmistaminen perustuu järjestelmän *monitorointiin*
- Esim. sosiaalisen median palvelussa
  - palvelun muistin ja prosessoriajan kulutusta
  - verkkoliikenteen määrää
  - sovelluksen eri sivujen vasteaikoja
  - kirjautuneiden käyttäjien määrää
  - luettujen ja lähetettyjen viestien määriä per käyttäjä
  - kirjautuneen käyttäjän sovelluksessa viettämää aikaa
- Monitoroidaan palvelimen yleisen toimivuuden lisäksi *käyttäjätason metriikoita* (engl. business level metrics)
- Jos suuria eroja aiempaan: **rollback edelliseen versioon**
  - esim. kirjautuneet käyttäjät eivät lähetä viestejä samaa määrää kuin keskimäärin normaalisti

- Uuden version toimivaksi varmistaminen perustuu järjestelmän *monitorointiin*
- Esim. sosiaalisen median palvelussa
  - palvelun muistin ja prosessoriajan kulutusta
  - verkkoliikenteen määrää
  - sovelluksen eri sivujen vasteaikoja
  - kirjautuneiden käyttäjien määrää
  - luettujen ja lähetettyjen viestien määriä per käyttäjä
  - kirjautuneen käyttäjän sovelluksessa viettämää aikaa
- Monitoroidaan palvelimen yleisen toimivuuden lisäksi *käyttäjätason metriikoita* (engl. business level metrics)
- Jos suuria eroja aiempaan: **rollback edelliseen versioon**
  - esim. kirjautuneet käyttäjät eivät lähetä viestejä samaa määrää kuin keskimäärin normaalisti
- Testauksen ja kaikkien tuotantoon vientiin liittyvän on syytä tapahtua automatisoidusti

Havainnoitavuus ja palvelutasotavoitteet

- Ennalta valitut mittarit kertovat, *että* jotain on vialla, mutta eivät välttämättä *miksi*

Havainnoitavuus ja palvelutasotavoitteet

- Ennalta valitut mittarit kertovat, *että* jotain on vialla, mutta eivät välttämättä *miksi*
- *Havainnoitavuus* (observability): sovellus tuottaa lokeja, metriikoita ja pyyntöjen kulkua kuvaavia jälkiä (traces)
  - avoin standardi OpenTelemetry

Havainnoitavuus ja palvelutasotavoitteet

- Ennalta valitut mittarit kertovat, *että* jotain on vialla, mutta eivät välttämättä *miksi*
- *Havainnoitavuus* (observability): sovellus tuottaa lokeja, metriikoita ja pyyntöjen kulkua kuvaavia jälkiä (traces)
  - avoin standardi OpenTelemetry
- Googlen Site Reliability Engineering: palvelutasotavoitteet (SLO)
  - esim. 99,9 % pyynnöistä onnistuu ja 95 % vastataan alle 300 ms:ssa
  - canary voidaan perua automaattisesti, jos uusi versio rikkoo tavoitteita

Virhebudjetti ja syyllistämätön jälkianalyysi

- *Virhebudjetti* (error budget): jos tavoite 99,9 %, saa 0,1 % pyynnöistä epäonnistua
  - budjettia jäljellä: uusia ominaisuuksia tuotantoon normaalisti
  - budjetti lopussa: painopiste luotettavuuden parantamiseen

Virhebudjetti ja syyllistämätön jälkianalyysi

- *Virhebudjetti* (error budget): jos tavoite 99,9 %, saa 0,1 % pyynnöistä epäonnistua
  - budjettia jäljellä: uusia ominaisuuksia tuotantoon normaalisti
  - budjetti lopussa: painopiste luotettavuuden parantamiseen
- Vakavan häiriön jälkeen *jälkianalyysi* (postmortem): mitä tapahtui, miksi, miten estetään jatkossa

Virhebudjetti ja syyllistämätön jälkianalyysi

- *Virhebudjetti* (error budget): jos tavoite 99,9 %, saa 0,1 % pyynnöistä epäonnistua
  - budjettia jäljellä: uusia ominaisuuksia tuotantoon normaalisti
  - budjetti lopussa: painopiste luotettavuuden parantamiseen
- Vakavan häiriön jälkeen *jälkianalyysi* (postmortem): mitä tapahtui, miksi, miten estetään jatkossa
- Analyysi on *syyllistämätön* (blameless)
  - virheet nähdään järjestelmän ja prosessien ongelmina, ei yksilöiden
  - jos virheistä rangaistaan, niitä piilotellaan eikä niistä opita
  - sama ajatus kuin psykologisessa turvallisuudessa

Feature flag

- *Feature flagien* (myös *feature toggle*) avulla voidaan canary releaset toteuttaa käyttämällä yhtä tuotantopalvelinta

Feature flag

- *Feature flagien* (myös *feature toggle*) avulla voidaan canary releaset toteuttaa käyttämällä yhtä tuotantopalvelinta
- Koodiin *ehtolauseita*: osa liikenteestä ohjataan vanhan toteutuksen sijaan testauksen alla olevaan toteutukseen

Feature flag

- *Feature flagien* (myös *feature toggle*) avulla voidaan canary releaset toteuttaa käyttämällä yhtä tuotantopalvelinta
- Koodiin *ehtolauseita*: osa liikenteestä ohjataan vanhan toteutuksen sijaan testauksen alla olevaan toteutukseen
- Esim. some-palvelussa feature flag: *osalle käytetään näytetään uuden algoritmin perusteella generoitu lista uutisia*

```
def recommended_news_generator(user):
    if is_in_canary_release(user):
        return experimental_recommendation_algorithm(user)
    else:
        return recommendation_algoritm(user)
```

Feature flagien soveltaminen

- Aluksi piilotetaan uusi ominaisuus käyttäjiltä feature flageilla
  - eli flag palauttaa vanhan version normaaleille käyttäjille

Feature flagien soveltaminen

- Aluksi piilotetaan uusi ominaisuus käyttäjiltä feature flageilla
  - eli flag palauttaa vanhan version normaaleille käyttäjille
- Sovellus kehittäjien mahdollista valita kumman version flag palauttaa

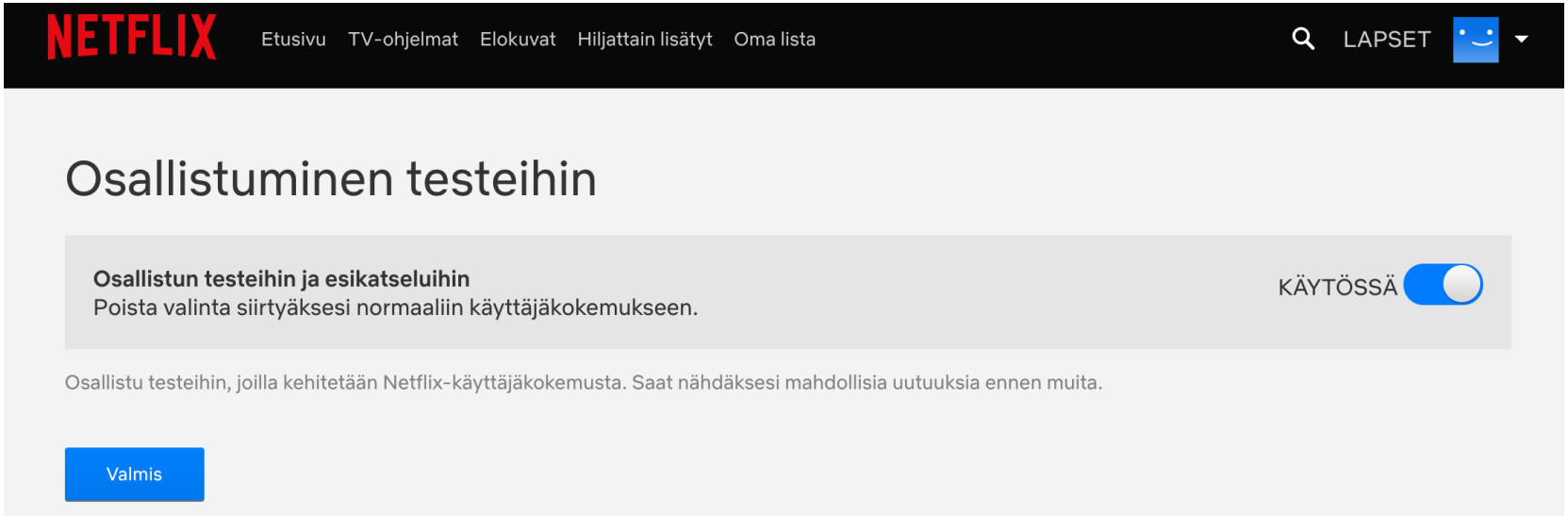
Feature flagien soveltaminen

- Aluksi piilotetaan uusi ominaisuus käyttäjiltä feature flageilla
  - eli flag palauttaa vanhan version normaaleille käyttäjille
- Sovellus kehittäjien mahdollista valita kumman version flag palauttaa
- Kun valmiina laajempaan testiin, julkaistaan esim.
  - ensin kehittäjäyrityksen omaan käyttöön
  - sitten osalle käyttäjistä canary releasena

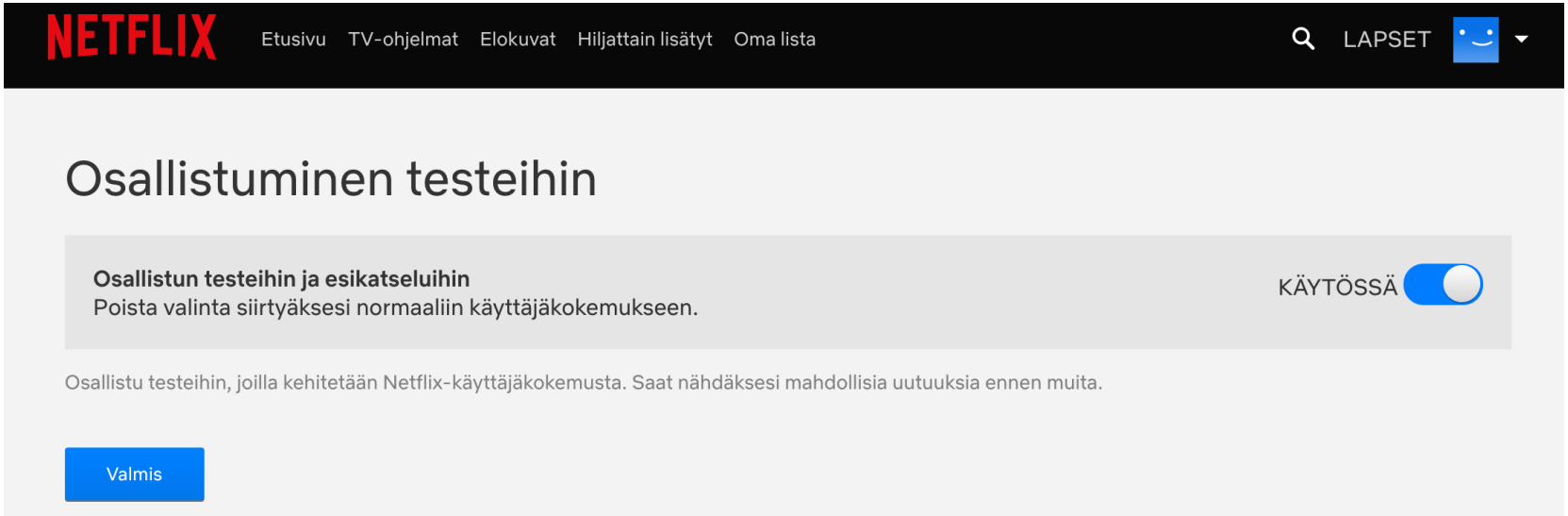
Feature flagien soveltaminen

- Aluksi piilotetaan uusi ominaisuus käyttäjiltä feature flageilla
  - eli flag palauttaa vanhan version normaaleille käyttäjille
- Sovellus kehittäjien mahdollista valita kumman version flag palauttaa
- Kun valmiina laajempaan testiin, julkaistaan esim.
  - ensin kehittäjäyrityksen omaan käyttöön
  - sitten osalle käyttäjistä canary releasena
- Lopulta feature flag ja vanha toteutus voidaan poistaa

- Suuret internetpalvelut soveltavat laajalti canary releaseihin ja feature flageihin perustuvaa kehitysmallia
  - Meta, Netflix, Google, GitHub, ...
  - Suomessa esim. Veikkaus

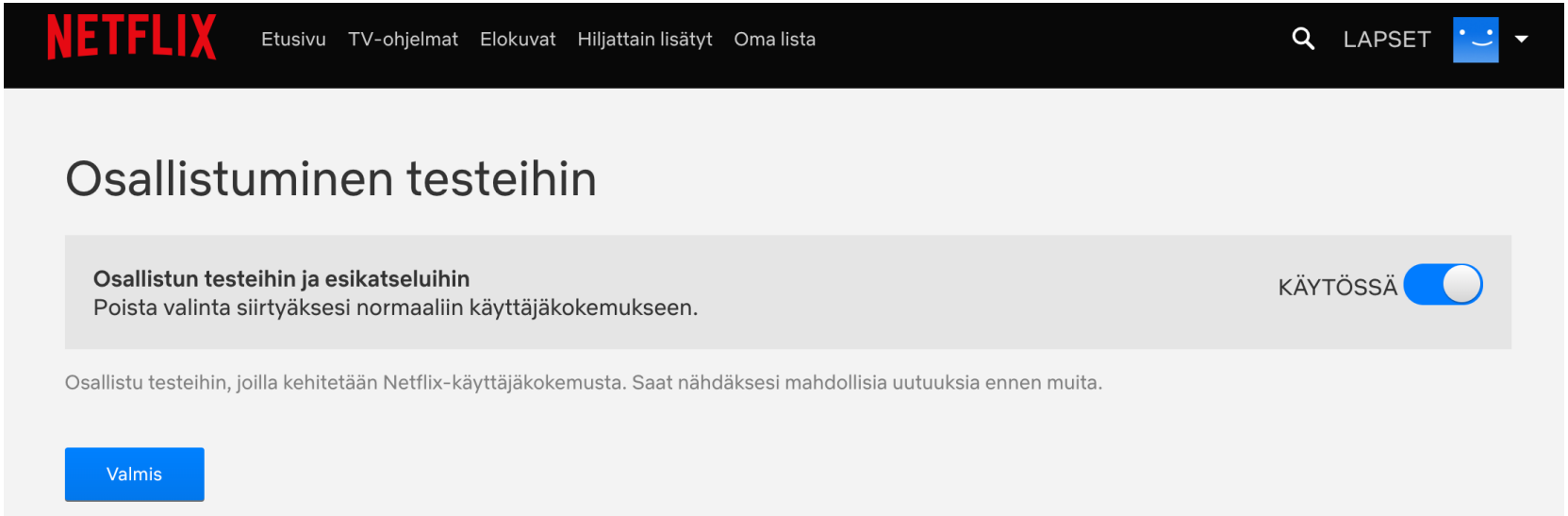


- Suuret internetpalvelut soveltavat laajalti canary releaseihin ja feature flageihin perustuvaa kehitysmallia
  - Meta, Netflix, Google, GitHub, ...
  - Suomessa esim. Veikkaus



- A/B-testaus: arvioidaan onko uusi toteutus parempi kuin vanha

- Suuret internetpalvelut soveltavat laajalti canary releaseihin ja feature flageihin perustuvaa kehitysmallia
  - Meta, Netflix, Google, GitHub, ...
  - Suomessa esim. Veikkaus



- A/B-testaus: arvioidaan onko uusi toteutus parempi kuin vanha
- Kerrallaan voi olla menossa useita kymmeniä A/B-testattavia eksperimenttejä